Machine Learning Project

Gian Luca Ascari (r1031238) - Matteo Naccarato (r1032714)

May 2025

1 Task 1

The three papers that influenced our work the most are:

- Bloembergen et al. [2], which provided insights into the relationship between evolutionary game theory and Multi-Agent Reinforcement Learning, as well as details on the replicator dynamics in Q-learning and lenient Q-learning.
- Shulman et al. [4], which allowed us to better understand the main components of the PPO Algorithm, helping us to take informed actions to improve the performance in tasks 3 and 4.
- Lucian Busoniu et al. [3], which offered a comprehensive overview of existing learning algorithms, highlighting their relative strengths and weaknesses, and communication methodologies in Multi-Agent Learning.

2 Task 2

2.1 Nash equilibria and Pareto Optimal states

Game	Nash Equilibria	Pareto Optimal State
Stag Hunt	(Stag, Stag), (Hare, Hare)	(Stag, Stag)
Subsidy Game	(S1, S1),(S2,S2)	(S1, S1)
Matching Pennies	Mixed Strategy. ((Prob(H)=0.5,Prob(T)=0.5), (Prob(H)=0.5,Prob(T)=0.5))	(H,H),(H,T),(T,H),(T,T)
Prisoner's Dilemma	(Defect, Defect)	(Cooperate, Cooperate)

Table 1: Nash equilibria and Pareto optimal states for different games.

2.2 Implement and Comment

Q-learning is a model-free reinforcement learning algorithm used to find the optimal action-selection policy for an agent in a given environment. It is based on the concept of Q-values (or quality values), which estimate the expected cumulative reward of taking a specific action in a given state and following the optimal policy thereafter.

A crucial part of the Q-learning algorithm is the strategy to balance exploitation (following what currently seems to be the best strategy to maximize reward) and exploration (exploring actions outside the greedy policy to determine whether better strategies exist). Some strategies to achieve this balance include ε-greedy, Boltzmann, and Lenient Boltzmann. In this section, we will apply these algorithms to the matrix games listed above and discuss the obtained results.

2.2.1 ε-greedy Q-learning

ε-greedy Q-learning is a reinforcement learning algorithm that balances exploration and exploitation based on the learning parameter ε.
Learning parameters:

- $\varepsilon \in (0, 1)$: with probability ε, the agent explores by randomly choosing an available action, and with probability $1 - \varepsilon$, it exploits by selecting the action with the highest Q-value.

We decided to decay ε over time, allowing the agent more exploration in the early stages and focusing more on the exploitation once the learned knowledge is more solid. We plot the policies histories of the two players (grey lines) and the mean trajectory of the probabilities of cooperation (blue line) from the starting state (green dot) to the ending state (red dot); this helps to grasp the evolution of the policies. The impact of the exploration rate ε can be observed in Figure 1 (with a minimum $\varepsilon = 0.1$) and Figure 2 (with a minimum $\varepsilon = 0.5$). A higher ε increases the likelihood of selecting actions at random, reducing exploitation of the learned policy. This effect is evident in the right-hand figure, where the probability of cooperation stabilizes at 0.25 (as indicated by the red dot), compared to 0.05 in the left-hand figure.

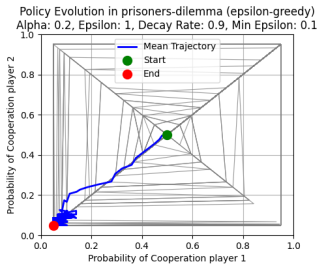


Figure 1: Prisoner's Dilemma - Epsilon Greedy - $min_\varepsilon = 0.1$

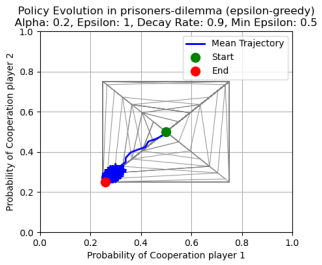


Figure 2: Prisoner's Dilemma - Epsilon Greedy - $min_\varepsilon = 0.5$

Here, we discuss the convergence to Nash Equilibrium or Pareto Optimality in the matrix games:

- Stag Hunt (Figure 3): The agents' behaviors highly depend on the exploration strategy. Observing the plot, we can see that the ε-greedy learning algorithm generates squares of different dimensions over time. This occurs due to the variation of ε after each Q-value update, starting from $\varepsilon = 1$ and decreasing to 0.1 with a decay rate of 0.9. Examining the learning process, we observe that both players gradually reduced their cooperation probabilities, going close to $(0, 0)$.
- Subsidy Game (Figure 4): Initially, both players exhibit moderate cooperation probabilities, but as long as the learning progresses, their cooperation levels decline towards $(0, 0)$.
- Matching Pennies (Figure 5): Since ε-greedy involves random exploration, especially at early stages, we see high variance in the learning trajectories. The plot perfectly shows how there is no convergence to a pure strategy, instead, the policies oscillate around the centre $(0.5, 0.5)$.
- Prisoner's Dilemma (Figure 6): Similarly to the Stag Hunt and the Subsidy Game, the policy initially varies, creating grey squares of different dimensions. However, it eventually converges to the Nash Equilibrium point of non-cooperation.

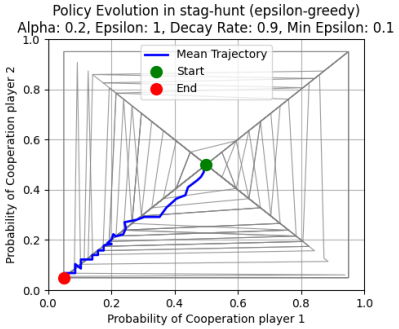


Figure 3: Stag Hunt - ε-greedy Q-learning Plot

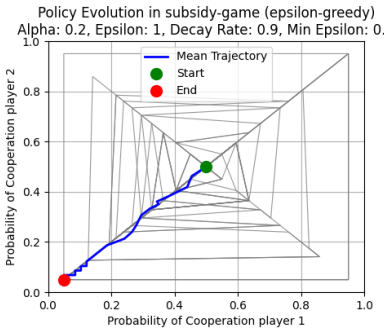


Figure 4: Subsidy Game - ε-greedy Q-learning Plot

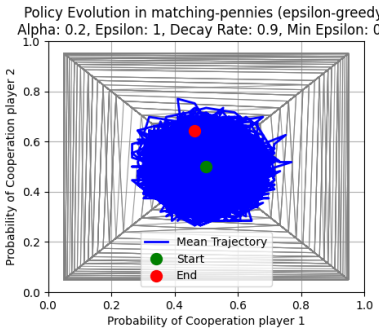


Figure 5: Matching Pennies - ε-greedy Q-learning Plot

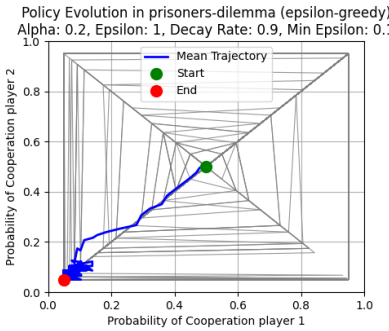


Figure 6: Prisoner's Dilemma - ε-greedy Q-learning Plot

2.2.2 Boltzmann Q-learning

Boltzmann Q-learning addresses the challenge of balancing exploration and exploitation by choosing actions with probability:

$$h(x,a) = \frac{e^{Q(x,a)/\tau}}{\sum_{\tilde{a}} e^{Q(x,\tilde{a})/\tau}}$$

Parameters:

- $\tau \in (0, \infty)$ is a hyperparameter of the algorithm called temperature. The higher the temperature value, the more the agent will explore; this is due to the fact that the actual Q values will have a smaller effect on the exponential. On the other hand, a lower value makes the choices go more towards exploitation, reaching a behavior more similar to the greedy strategy.

The effect of τ can be seen in the contrast between Figure 7 and Figure 8. The former shows the behavior with a higher value: here, the agents have an initial exploration phase with a balanced choice of actions (curves moving to the center), after which a more exploitation-oriented phase occurs, leveraging what seems the best action, but always keeping a bit of exploration (curves never reaching the edges). The ladder shows the effect of a lower value of τ, which leads agents to take more firm decisions about what action to take, highly restricting exploration.

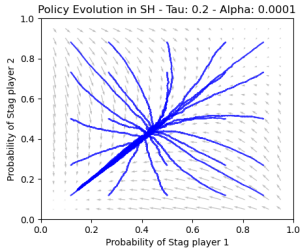


Figure 7: Stag Hunt - Boltzmann Q-learning with $\tau = 0.2$

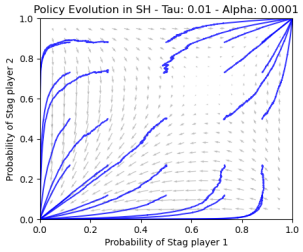


Figure 8: Stag Hunt - Boltzmann Q-learning with $\tau = 0.01$

Convergence to Nash Equilibrium or Pareto Optimal states:

- Stag Hunt: As shown in Figure 9, this algorithm always converges to a Nash Equilibrium in the Stag Hunt game. In general, it follows the replicator dynamics: if both players start with a policy favoring collaboration on the Stag, they can exploit it and reach the Pareto Optimal state. Otherwise, they both adopt a more conservative strategy, choosing Hare to secure a guaranteed reward.

- Prisoner's Dilemma: As shown in Figure 10, in the Prisoner's Dilemma, the algorithm usually follows trajectories similar to the replicator dynamics, tending in most cases to converge to the only Nash Equilibrium (Defect, Defect). However, when the initial Q-values are heavily biased toward the Collaboration action, the softmax function becomes dominated by it, preventing sufficient exploration of the Defect action and thus failing to converge to the Nash Equilibrium. This

discrepancy between the algorithm's behavior and the replicator dynamics is explained by Kaisers and Tuyls [1].

- Matching Pennies: This game has no pure Nash Equilibrium; the equilibrium would be a mixed strategy where the probability of choosing Heads or Tails is 1/2. As shown in Figure 11, this is reflected both in the replicator dynamics, which form a loop, and in the algorithm's actual results, which follow similar dynamics.
- Subsidy Game: Here a similar analysis applies to the Prisoner's Dilemma one. As shown in Figure 12, in most cases, the algorithm follows the replicator dynamics and converges to the Nash Equilibrium. However, when the initial policies are highly biased, the softmax function becomes dominated by action S_2, causing the agents to converge to a suboptimal policy. This behavior is discussed by Kaisers and Tuyls [1]

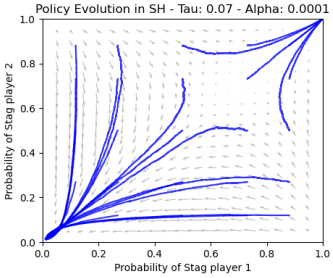


Figure 9: Stag Hunt - Boltzmann Q-learning

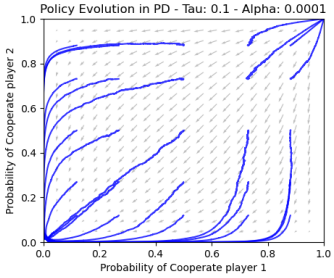


Figure 10: Prisoner's Dilemma - Boltzmann Q-learning

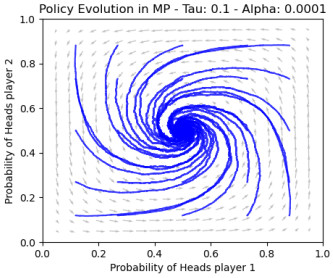


Figure 11: Matching Pennies - Boltzmann Q-learning

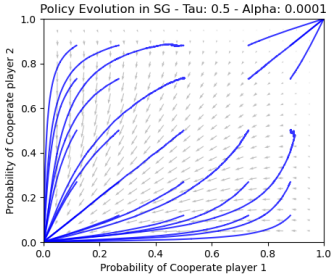


Figure 12: Subsidy Game - Boltzmann Q-learning

2.2.3 Lenient Boltzmann Q-learning

Lenient Boltzmann Q-learning is a reinforcement learning algorithm that balances exploration using a k parameter. In multi-agent scenarios, agents can be penalised for initially selecting a bad action, indeed, this issue is addressed by making the process more forgiving of initial mistakes, introducing a leniency parameter.
Learning parameters:

- $k \in (1, \infty)$: it represents how lenient an agent is after receiving low rewards. In fact, during the Q-value update, the agent will take into account the last k rewards received for that action and will use the best one to perform the FAQ-value update in the formula $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \frac{1}{P(a_t)} * \alpha \left[r_{max_k} - Q(s_t, a_t)\right]$. A low k produces similar results to Boltzmann Q-learning, while a higher k helps the agents to increase their leniency, resulting in stronger cooperation actions. This comparison is shown in Figure 13 and Figure 14.

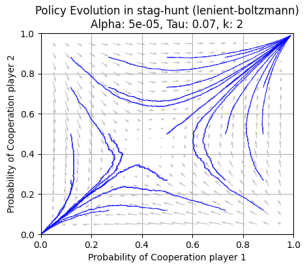


Figure 13: Stag Hunt - Lenient Boltzmann - $k = 2$

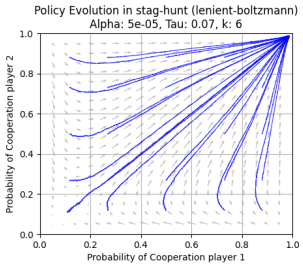


Figure 14: Stag Hunt - Lenient Boltzmann - $k = 6$

Convergence to Nash Equilibrium or Pareto Optimality in the following matrix games:

- Stag Hunt (Figure 15): The Lenient-Boltzmann exploration approach is strongly evident here. Indeed, it encourages agents to gradually prefer higher-reward actions. This leads to the convergence at the full cooperation $(Stag, Stag)$ state, which represents both the Nash Equilibrium and the Pareto Optimal State.
- Subsidy Game (Figure 16): Using $k = 3$, we can appreciate the difference from the Boltzmann Q-Learning version since the cooperation area is more evident. Nevertheless, the non-cooperation convergence is still dominant.
- Matching Pennies (Figure 17): Here, the trajectories show structured exploration moving between regions before converging towards the centre. Indeed, the centre of the plot represents a mixed-strategy equilibrium where both players choose actions with equal probability. Moreover, the evolution of the trajectories is also reflected in the replicator dynamics plotted in the background.
- Prisoner's Dilemma (Figure 18): In this game, the leniency parameter does not influence the results in a significant way. Indeed, the two agents converge to the non-collaborative Nash Equilibria, since the cooperation is a non-convenient action.

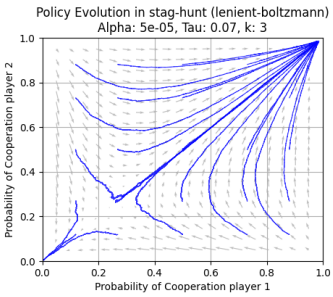


Figure 15: Stag Hunt - Lenient Boltzmann Q-learning

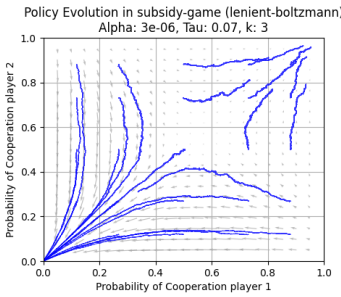


Figure 16: Subsidy Game - Lenient Boltzmann Q-learning

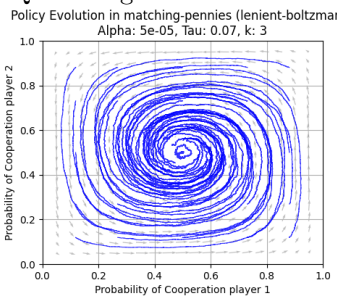


Figure 17: Matching Pennies - Lenient Boltzmann Q-learning

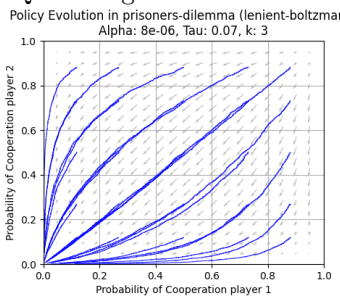


Figure 18: Prisoner's Dilemma - Lenient Boltzmann Q-learning

2.2.4 Algorithms comparison

Algorithm	**Exploration Mechanism**	**Strengths**	**Weaknesses**
ε-Greedy	Randomly selects an action with probability ε, otherwise picks the highest Q-value.	Easy to implement	Can get stuck in suboptimal policies.
Boltzmann	Selects actions probabilistically based on their Q-values using a softmax function.	More structured exploration.	Can still settle in local optima.
Lenient-Boltzmann	Similar to Boltzmann strategy, but it allows more leniency for suboptimal actions.	Reduces suboptimal convergence by encouraging gradual exploration.	More complex to implement due to the tune of the leniency function.

Table 2: Comparison of Learning Algorithms

3 Task 3

Real world applications happen in much more complex environments than the ones represented in matrix games, such environments require learning algorithms that are able to generalize, reusing the insights of the seen scenarios in unseen ones. In this task we use one of these algorithms (Proximal Policy Optimization [4]) to train an agent in the Knights Archers Zombies Pettingzoo environment [1].

Proximal Policy Optimization (PPO) is a policy gradient algorithm that combines the stability of trust region methods with the simplicity of first-order optimization. It updates the policy by maximizing a clipped surrogate objective that limits how much the policy can change at each step, thereby preventing large, destabilizing updates.

3.1 Initial setup

At the beginning of our experiments, we set the hyperparameters of PPO to the following:

Parameter	**Value**
Batch size	**512**
Num epochs	**1**
Learning rate	**1e-4**
γ	**0.99**
λ	**1**
Clip param	**0.3**

Table 3: Initial training parameters

As input tensor we used the plain vectorized observation array given by the PettingZoo environment [2]. The neural network was structured with 2 fully connected layers that counted 64 neurons each.

We trained our agent for 1000 iterations, and the resulting policy showed (in our local testing) an average reward of 2.46, slightly better than random play (2.03). We show in Figure 19 the trajectories of average rewards and episode length produced by the training.

[1] `https://pettingzoo.farama.org/environments/butterfly/knights_archers_zombies/`

[2] `https://pettingzoo.farama.org/environments/butterfly/knights_archers_zombies/#vectorized-default`

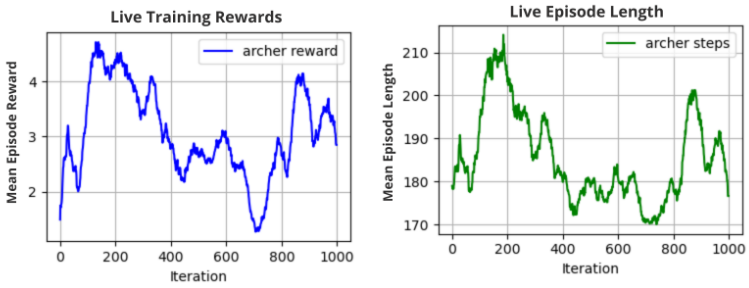


Figure 19: Training curves of a single agent with the base setup

From these plots it is easy to see that the agent is struggling to learn significant insights from the environment, with a policy that does not show any significant overall improvement trend.

3.2 Batch size and number of epochs

Reading the work of Papini et al. [5] on how to obtain safer policy improvements, we understood that larger batch sizes would have allowed us to have more meaningful and consistent updates. We therefore decided to increase our training batch size to 4000. Furthermore, leveraging the ability of PPO to perform more epochs on the same batch of data, that as discussed by Schulmann et al. [4] in the original PPO paper is allowed by the clipped surrogate objective), we decided to set the number of epochs per batch to 10.

With this updated setup we obtained more consistent training curves (as visible in Figure 20) and an average reward in local testing of 19.58.

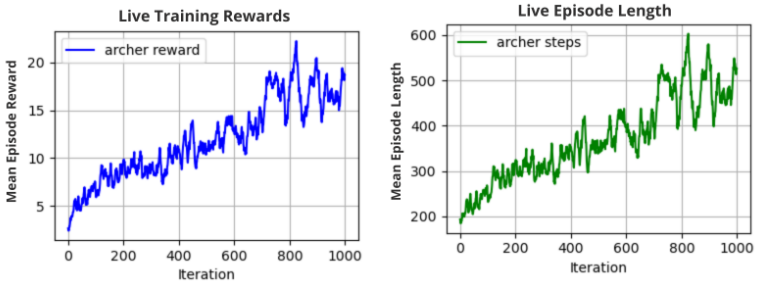


Figure 20: Single agent, updated setup with batch size 4000 and num SGD iter 10

3.3 Lambda and clip parameter

Among the effects of the setup update described in the previous section, it is also noticeable a significant increase of the variance in the training. To contrast this effect, we decided to leverage the values of λ and of the clip parameter.

The parameter λ is part of the Generalized Advantage Estimation (GAE) method, which PPO uses to estimate the advantage function. The advantage function measures how much better taking a specific action in a given state is compared to the average action in that state, and it plays a key role in computing the surrogate loss. As discussed by Schulman et al. [6], λ controls the trade-off between high bias and low variance (when set to 0) and low bias and high variance (when set to 1). To strike a good balance, we set this parameter to the commonly used value of 0.95, which helps reduce the effect of variance while maintaining reasonable bias.

The clip parameter is used in computing the surrogate loss function to prevent large, destabilizing policy updates. It limits how much the probability of taking a given action can change between successive policies. Following the findings of the original PPO paper [4], we decreased this value to 0.2, which showed to be effective for achieving more stable learning.

These additional updates helped us further improve the results, obtaining an average test reward of 39.38 and more stable learning curves as visible in Figure 21.

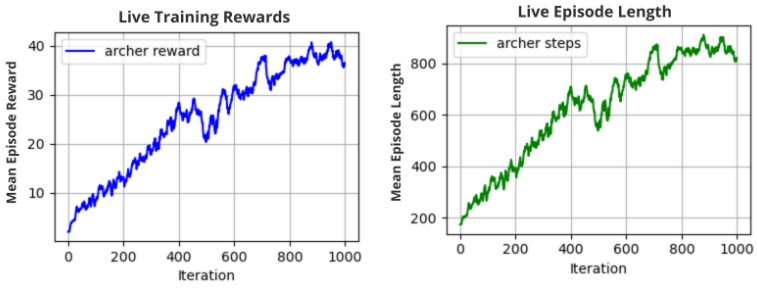


Figure 21: Single agent, updated setup with lambda 0.95 and clip parameter 0.2

3.4 Custom features

Observing the behavior of our agent, we noticed that even if it was able to achieve good scores, its shooting accuracy could have still been improved. In order to make it easier to learn how to correctly aim at zombies we added custom features that computed the angle differences between where the archer was aiming and the position of the zombies. This update brought a slight performance improvement leading to an average test reward of 44.79 and the training curves shown in Figure 22.

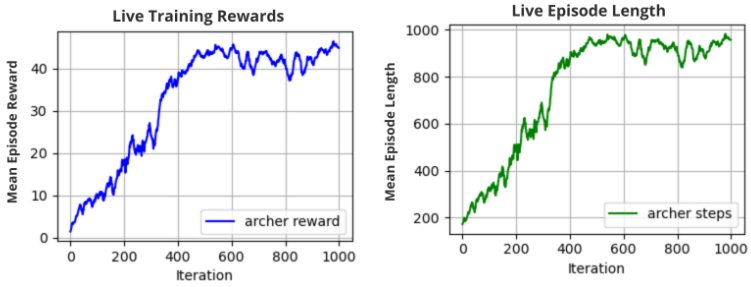


Figure 22: Single agent, updated setup with custom features

3.5 Dynamic learning rate

With the reached training setup the agent was consistently able to reach an average episode length close to 1000, the maximum allowed during training. However, the learning curves plotted in the previous section show that, once such level was reached, the trajectories saw a significant variance and were not able to stabilize in an optimal state of affairs.

We inferred that this could be due to large policy steps that made it jump from one suboptimal state to the other. To contrast this effect we decided to apply a dynamic learning rate, that over 2000 iterations would decrease from 3e-4 to 1e-6. This allowed us to obtain a final improvement on the average reward to 48.01 and the learning curves shown in Figure 23.

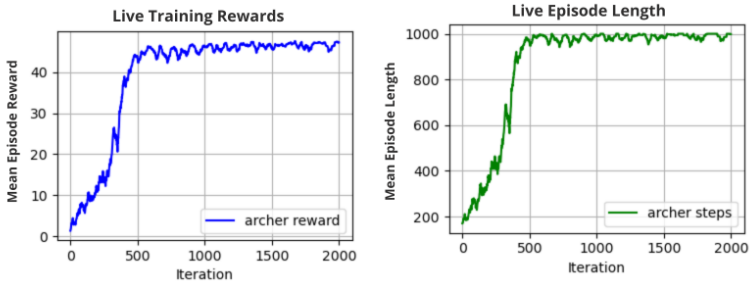


Figure 23: Single agent final setup with dynamic learning rate

4 Task 4

4.1 Discussion

Going from a single-agent version to a multi-agent one requires some adaptations to get a valuable result. Two approaches can be used to make this transition: *independent learning* and *joint-action learning*. The former requires the agents to mutually ignore each other, considering the interaction with the other agents as noise. In the latter, on the other side, the learners take the presence of the other agents into account by learning in the space of joint actions, rather than in their individual action space only. [2]. KAZ with multiple archers is a cooperative environment where the agents play in a coordinated way to maximise the rewards. On this topic, three coordination methodologies can be identified [3]:

- Coordination-Free: Q-learning avoids the coordination problem by assuming that the optimal joint actions are unique to which agents can converge without coordination.
- Coordination-Based: The goal is to actively coordinate agents. For example, a coordination graph can be used to represent which agents have to coordinate with. In this case, the global Q-function is divided into local Q-functions among linked agents.
  e.g. $Q_t(a_1, a_2, a_3) = Q_1(a_1, a_2) + Q_2(a_2, a_3)$ means that a_2 actions influence both a_1 and a_3.
- Indirect Communication: There is no explicit message exchange, but the agents learn what the others will do and consequently behave. This leads to a biased action selection towards actions that are likely to result in good rewards, steering the agents towards coordinated action selection.

A fundamental distinction in multi-agent environments regards the category of the agents. Indeed, multi-agent KAZ is an example of an environment with homogeneous agents, since all the agents share the same capabilities and characteristics (such as actions, observations, and rewards). We can identify two categories of homogeneous agents: *weakly*, and *strongly* homogeneous agents. In the former, the agents move in the environment using the same set of actions and receive the same rewards for performing similar actions, but may have different roles, policies or goals. In the strong version of it, instead, all agents act identically under the optimal policy when in identical conditions [8]. In KAZ, for example, the agents are considered weakly homogeneous if they have different roles, such as covering the left and right parts independently.

4.2 Multi-Agent KAZ RL in Practice

On the one hand, our initial idea, after having acquired the communication knowledge among agents, was to implement a strategy that splits the game field into left and right (or top and bottom). This approach is considered *cheap talk*, as each agent stays in its part of the game without directly reacting to the other agent's actions. While this serves as a starting point, it does not take into account the distribution of zombies on the game field. Indeed, to enhance coordination, electing a captain (e.g. $archer_0$) is a valid approach. In this case, the role of the captain is to decide the primary decision, such that the other agent reacts accordingly (e.g. by focusing on the most dangerous area). If the captain has access to additional information, this scenario can be identified as a *signaling game*, where the sender observes some private information and sends a message that the receiver interprets before taking action.

On the other hand, we decided to set up a hybrid feature observation space, as described in subsection 3.4, and to tune the PPO configuration parameters following a similar approach of the one described in section 3. The following table shows the final version of the tuned parameters for the multi-agent version:

Parameter	**Value**
Batch size	**8000**
Num SGD iterations	**15**
Dynamic Learning rate Values	**3e-4** ; **1e-6**
γ	**0.99**
λ	**0.95**
Clip param	**0.2**
Iterations	**1000**

Table 4: Initial training parameters

Using this configuration, we implemented an indirect communication with centralised training, since every agent can see other agents' states. We noticed that the agents learnt to communicate by themselves by looking at the full observation space and to act optimally. The learnt policies involved positioning one archer more to the left and the other more to the right, splitting the game field in a clever way. Additionally, the archers moved forward when no zombies were present, allowing them to get closer to the zombie spawn line to shoot incoming zombies more quickly. However, the zombie generation frequency was not sufficiently challenging - after a few seconds, both archers targeted the same zombie as it was the only one on the field. As a result of the training process shown in the following plot, we got an average reward of 48.63.

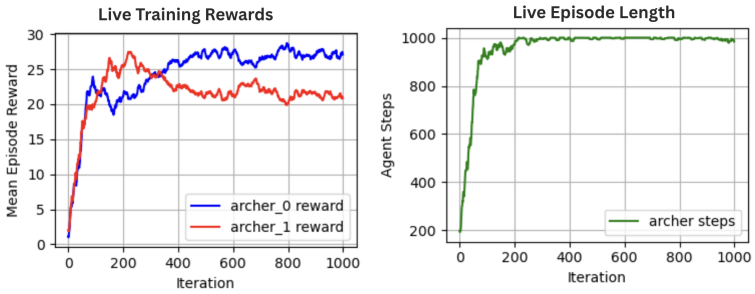


Figure 24: Multi agent training curves with 5

5 Report on the spent time

As requested we kept track of the time spent by each of us to perform this project and the results are visible in Table 5, we tried to be as precise as possible logging into a common table the time spent each time we worked on it. Some minor errors, especially in the divisions in tasks are possible, however this should be a rather accurate count of the hours. We added a category "general" for all the necessary work not related to any specific task, like setting up the SSH configuration to connect to the departmental machines, the revisions of the report before submission etc.

Task	Total	Gian Luca Ascari	Matteo Naccarato
Task 1	14	7	7
Task 2	30	18	12
Task 3	72	40	32
Task 4	15	1	14
General	27	14	13
Total	**157**	**80**	**77**

Table 5: Time spent and division of the tasks

6 Submission folder

The code used in the scope of this project can be found on the departmental computers in the submission folder of Gian Luca Ascari (r1031238): /cw/lvs/NoCsBack/vakken/H0T25A/ml-project/r1031238.

References

[1] M. Kaisers and K. Tuyls, "Frequency Adjusted Multiagent Q-Learning," in Proceedings of the Ninth International Conference on Autonomous Agents and Multi-Agent Systems (AAMAS 2010)

[2] Daan Bloembergen et al. "Evolutionary Dynamics of Multi-Agent Learning: A Survey". In: J. Artif. Intell. Res. (JAIR) 53 (2015), pp. 659–697.

[3] Lucian Busoniu, Robert Babuska, and Bart De Schutter. "A Comprehensive Survey of Multiagent Reinforcement Learning". In: IEEE Trans. Systems, Man, and Cybernetics, Part C 38.2 (2008).

[4] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. "Proximal Policy Optimization Algorithms". In: *arXiv preprint arXiv:1707.06347* (2017). URL: https://arxiv.org/abs/1707.06347.

[5] Matteo Papini, Matteo Pirotta, and Marcello Restelli. "Adaptive Batch Size for Safe Policy Gradients". In: *Advances in Neural Information Processing Systems*, vol. 30, 2017. URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/ea6b2efbdd4255a9f1b3bbc6399b58f4-Paper.pdf.

[6] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. "High-Dimensional Continuous Control Using Generalized Advantage Estimation". In: *arXiv*, 2018. URL: https://arxiv.org/abs/1506.02438.

[7] Yoav Shoham and Kevin Leyton-Brown. Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations. Cambridge University Press, 2009. http://www.masfoundations.org/mas.pdf

[8] Lukas Schäfer Stefano V. Albrecht Filippos Christianos. Multi-Agent Reinforcement Learning: Foundations and Modern Approaches. MIT Press, 2024. https://www.marl-book.com